CST8508 – Machine Vision

Webcam-Based Person Detection, Tracking, and Centering

Final Project Presentation

Hye Ran Yoo 041145212
Peng Wang 041107730

April 2026

1. Project Overview

What does this system do?

Presenter: Peng Wang

- Real-time person detection & tracking system using a laptop webcam
- Detects persons using mmdetection’s pre-trained RTMDet-s model
- Tracks detected persons with ByteTrack (stable unique IDs)
- Selects the largest bounding box when multiple persons detected
- Calculates offset between person center and frame center
- Displays direction messages: left, right, up, down + pixel amount
- Provides a cropped “Person Focus” zoomed view following the target
- Simulates a virtual pan-tilt camera that auto-follows a person

2. System Architecture

End-to-End Pipeline Diagram

Presenter: Peng Wang

Input

640×480 webcam stream via DirectShow (Windows)

Detection

RTMDet-s: CSPNeXt backbone, 44.6% AP, person class (label=0), conf ≥ 0.5

Tracking

ByteTrack: IoU-based association, handles occlusion, stable IDs

Output

Full View (annotated) + Person Focus (cropped 480×480 zoom)

3. Environment Setup

Software & Dependencies

Presenter: Peng Wang

Installation Commands

```
conda create -p mv_env_py311 python=3.11

pip install opencv-python numpy supervision
pip install openmim setuptools==69.5.1

pip install torch==2.2.2 torchvision==0.17.2 \
    --index-url
https://download.pytorch.org/whl/cpu

pip install mmcv==2.2.0   # prebuilt wheel
(CPU)
pip install mmengine mmdet
```

Key Dependencies

Python	3.11.15 (3.13 incompatible with MMCV)
PyTorch	2.2.2+cpu / torchvision 0.17.2+cpu
MMCV	2.2.0 (prebuilt wheel)
mmdetection	3.2.0 / mmengine 0.10.7
supervision	0.27.0 ByteTrack tracker (CPU)
OpenCV	4.13.0 Webcam capture & display
NumPy	1.26.4 Array operations

4. Detection Module – RTMDet-s

Pre-trained Object Detector from mmdetection

Presenter: Hye Ran Yoo

What is RTMDet-s?

- Pre-trained AI model from mmdetection library
- RTMDet-s = Real-Time Detection, Small version
- Already trained to detect 80 types of objects (COCO dataset)
- We only use class label 0 = "person"
- Confidence threshold: 0.5 (only keep detections ≥ 50% sure)

Why RTMDet-s?

- Fast enough to run on laptop CPU in real-time
- Small & lightweight model (no GPU required)
- Recommended by the professor's project guide

Code: detect_persons()

```
def detect_persons(inferencer, frame, config):
    result = inferencer(
        frame, show=False,
        no_save_vis=True,
        no_save_pred=True,
        print_result=False
    )
    pred = result['predictions'][0]
    bboxes = np.array(pred['bboxes'])
    scores = np.array(pred['scores'])
    labels = np.array(pred['labels'])

    # Filter: person class only + confidence
    person_id = config["person_class_id"]  # 0
    threshold = config["confidence_threshold"]
    mask = (labels == person_id) & \
           (scores >= threshold)
    return bboxes[mask], scores[mask], labels[mask]
```

5. Tracking Module – ByteTrack

Multi-Object Tracking with Stable IDs

Presenter: Hye Ran Yoo

How ByteTrack Works

- Two-stage data association algorithm
- Stage 1: Match HIGH confidence boxes to tracks
- Stage 2: Match LOW confidence boxes (recover misses)
- Uses IoU (Intersection over Union) for matching
- Assigns persistent unique ID per person
- Handles occlusion & temporary disappearance
- Gating mechanism filters redundant detections

Code: init & track

```
tracker = sv.ByteTrack(
    track_activation_threshold=0.25, # detection confidence: new?
    lost_track_buffer=30,
    minimum_matching_threshold=0.8,  # position overlap: same?
    frame_rate=30
)
tracked = tracker.update_with_detections(dets)
```

Key Parameters

track_activation_threshold	**0.25**	Min conf to start new track
lost_track_buffer	**30**	Frames to keep lost track alive
minimum_matching_threshold	**0.8**	Min IoU for matching
frame_rate	**30**	Expected FPS for motion prediction

6. Largest Bounding Box Selection

Focus on the most prominent person

Presenter: Hye Ran Yoo

- When multiple persons detected, select the LARGEST bounding box - The largest box is the closest person
- Area = (x2 − x1) × (y2 − y1) → np.argmax(areas)
- Assumption: largest box = closest / most prominent person
- Selected person marked with RED box + 'TARGET ID:X' label
- All other persons shown with GREEN boxes

```
def select_largest_person(detections):
    if len(detections) == 0:
        return None, None
    widths  = detections.xyxy[:, 2] - detections.xyxy[:, 0]
    heights = detections.xyxy[:, 3] - detections.xyxy[:, 1]
    areas = widths * heights
    idx = np.argmax(areas)
    bbox = detections.xyxy[idx]
    tracker_id = detections.tracker_id[idx]
    return bbox, tracker_id
```

7. Camera Centering Logic

Direction & Movement Amount Calculation

Presenter: Hye Ran Yoo

Center Calculation

- Image center: (width/2, height/2)
- Person center: (x1 + box_w/2, y1 + box_h/2)
- dx = person_cx − img_cx (+ = right)
- dy = person_cy − img_cy (+ = below)
- Tolerance zone: 5% of frame size → Prevents jittery messages

▶ If person LEFT of center → GREEN msg

▶ If person RIGHT of center → GREEN msg

▶ If person ABOVE center → RED msg

▶ If person BELOW center → RED msg

▶ Each msg includes pixel offset amount

Code: calculate_centering()

```
def calculate_centering(bbox, w, h, tol_ratio):
    img_cx, img_cy = w / 2, h / 2
    x1, y1, x2, y2 = bbox
    person_cx = x1 + (x2-x1) / 2
    person_cy = y1 + (y2-y1) / 2
    dx = person_cx - img_cx
    dy = person_cy - img_cy
    tol_x = w * tol_ratio
    tol_y = h * tol_ratio

    directions = []
    if person_cx < img_cx - tol_x:
        directions.append(f"move left ({abs(dx):.0f}px)")
    elif person_cx > img_cx + tol_x:
        directions.append(f"move right ({abs(dx):.0f}px)")
    if person_cy < img_cy - tol_y:
        directions.append(f"move up ({abs(dy):.0f}px)")
    elif person_cy > img_cy + tol_y:
        directions.append(f"move down ({abs(dy):.0f}px)")

    return directions, dx, dy, len(directions)==0
```

8. Crop & Center View

Virtual Pan-Tilt Camera Simulation

Presenter: Hye Ran Yoo

- Crops the video frame around the detected person’s bounding box
- Configurable padding ratio (crop_padding_ratio = 0.8)
- Resized to fixed 480×480 display size
- Simulates a virtual camera that auto-follows the target person
- Overlays: Target ID, offset values, direction guidance

```
def crop_centered_view(frame, bbox, config):
    h, w = frame.shape[:2]
    x1, y1, x2, y2 = bbox
    pad = config["crop_padding_ratio"]  # 0.8
    pad_w = int((x2-x1) * pad)
    pad_h = int((y2-y1) * pad)
    crop = frame[max(0,int(y1-pad_h)):min(h,int(y2+pad_h)),
                 max(0,int(x1-pad_w)):min(w,int(x2+pad_w))]
    return cv2.resize(crop, (480, 480))
```

9. Configuration Management

External config.json for **Easy Parameter Tuning**

Presenter: Hye Ran Yoo

```
{
    "model_name": "rtmdet_s_8xb32-300e_coco",
    "device": "cpu",
    "person_class_id": 0,
    "confidence_threshold": 0.5,
    "camera_id": 0,
    "frame_width": 640,
    "frame_height": 480,
    "center_tolerance_ratio": 0.05,
    "crop_padding_ratio": 0.8,
    "crop_display_size": [480, 480],
    "bytetrack": {
        "track_activation_threshold": 0.25,
        "lost_track_buffer": 30,
        "minimum_matching_threshold": 0.8,
        "frame_rate": 30
    }
}
```

config.json

- Separates Code & Settings
- Enables Rapid Threshold Tuning for Inference

Parameters

- confidence_threshold = 0.5 → Min. score for **valid detection**
- center_tolerance_ratio = 0.05 → 5% dead-zone to prevent camera jitter
- crop_padding_ratio = 0.8 → 80% margin around target crop
- track_activation_threshold = 0.25 → Min confidence to start new track
- lost_track_buffer = 30 → Frames to maintain ID **during occlusion**
- minimum_matching_threshold = 0.8 → Min. IoU for **track association**

10. Main Video Loop

Code Walkthrough – video_stream()

Presenter: Hye Ran Yoo

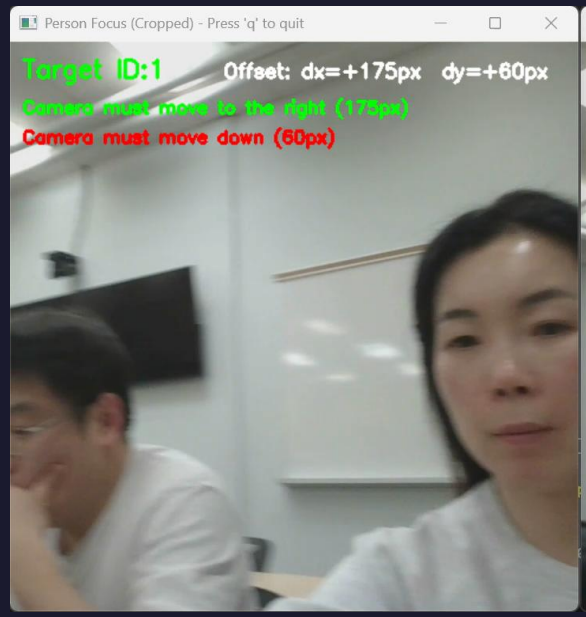
```
def video_stream():
    config = load_config()                # Load config.json
    inferencer = init_detector(config)    # Load RTMDet-s
    tracker = init_tracker(config)        # Init ByteTrack

    cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)  # Open webcam

    while True:
        ret, frame = cap.read()
        # 1) Detect persons
        bboxes, scores, labels = detect_persons(inferencer, frame, config)
        # 2) Track across frames
        tracked = track_persons(tracker, bboxes, scores, labels)
        # 3) Draw all person boxes (green)
        for i in range(len(tracked)):
            cv2.rectangle(frame, ...)   # Green box + ID
        # 4) Select largest person : Overwrite the biggest person with a Red box
        target_bbox, target_id = select_largest_person(tracked)
        # 5) Calculate centering
        directions, dx, dy, centered = calculate_centering(...)
        # 6) Draw: red TARGET box, crosshair, direction msgs
        # 7) Generate cropped "Person Focus" view
        cropped = crop_centered_view(clean_frame, target_bbox, config)
        cv2.imshow("Full View", frame)
        cv2.imshow("Person Focus", cropped)
```

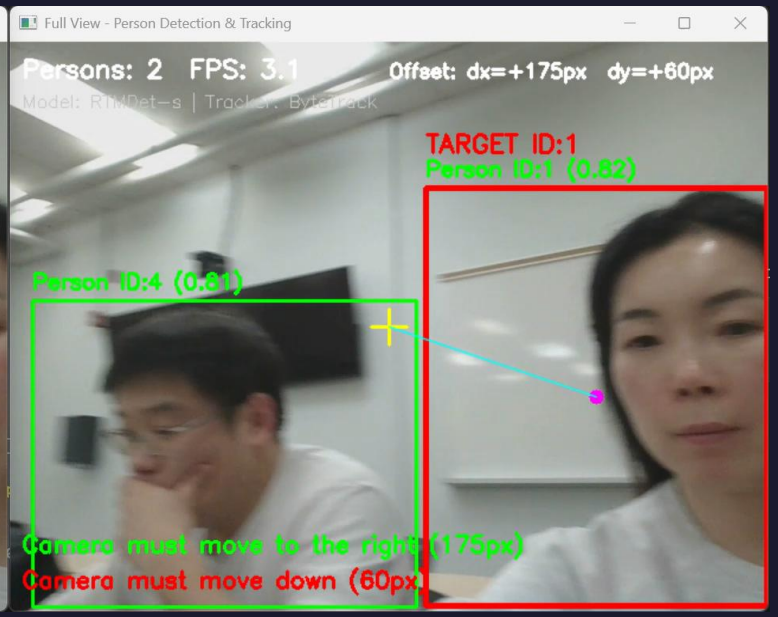
11. Live Demo / Demo Video

System in Action

Presenter: Hye Ran Yoo

Cropped View





Main Camera View

- Two simultaneous windows: Full View + Person Focus (Cropped)
- Real-time bounding boxes with tracking IDs
- Direction messages with pixel offsets update per frame
- Automatic target switching when largest person changes

12. Results – Visual Annotations

What appears on screen

Presenter: Peng Wang

■ **Green bounding boxes**	Person ID:X (confidence) for all tracked persons
■ **Red bounding box**	TARGET ID:X on the largest (selected) person
+ Yellow crosshair	Frame center marker
● **Magenta dot**	Person's bounding box center
— Cyan line	Connects frame center to person center
▶ **Orange direction msgs**	Camera must move left/right/up/down (Xpx)
dx, dy values	Pixel offset displayed in top-right corner
HUD overlay	Person count, FPS, model name, tracker name

→ Insert screenshot of running system here (Figure 1 from report)

12. Challenges & Lessons Learned

What we faced & what we gained

Presenter: Peng Wang

Challenges

- mmdetection 3.2.0 Windows install
  - C++ extensions require exact compiler & version matching
- Python 3.13 breaks MMCV 2.2.0
  - Pre-built wheels only exist for Python <=3.11
- Camera warm-up: dark first frames
  - Auto-exposure needs 10-20 frames to stabilize
- Confidence threshold balancing
  - 0.3 = false positives, 0.7 = misses distant persons, 0.5 optimal

Lessons Learned

- End-to-end CV pipeline thinking
  - Each module's output feeds the next; one failure cascades
- Version matrix is critical
  - PyTorch 2.2.2 - MMCV 2.2.0 - mmdet 3.2.0 must match exactly
- Config/code separation
  - External JSON lets you tune thresholds without touching code
- Real-time debugging needs HUD
  - Can't set breakpoints in video loop; visual overlay is essential

13. Tech Stack

Verified package versions from project environment

Presenter: Peng Wang

Package	Version	Role in Project
Python	3.11.15	Runtime (3.13 incompatible with MMCV C++ extensions)
PyTorch	2.2.2+cpu	Deep learning framework for RTMDet-s inference
torchvision	0.17.2+cpu	Image transforms & model utilities
MMCV	2.2.0	OpenMMLab foundation library (pre-built CPU wheel)
mmdetection	3.2.0	Object detection toolbox providing RTMDet-s model
mmengine	0.10.7	OpenMMLab training/inference engine
supervision	0.27.0	ByteTrack multi-object tracker implementation
OpenCV	4.13.0	Webcam capture, frame display, image drawing
NumPy	1.26.4	Array operations for bbox math & filtering

14. Evaluation

System Performance & Assessment

Strengths

- RTMDet-s: 44.6% AP on COCO, CPU real-time
  - Pre-trained model, no GPU or fine-tuning required
- ByteTrack (supervision 0.27.0): stable IDs
  - Two-stage IoU association + 30-frame occlusion buffer
- 5% dead zone prevents direction jitter
  - Only fires when offset exceeds frame_size x 0.05
- Modular architecture
  - Can swap RTMDet-s for YOLOv8 or ByteTrack for DeepSORT
- Dual-window: Full View + Person Focus
  - Context + detail simultaneously, 640x480 + 480x480

Limitations

- CPU-only: ~10-15 FPS
  - GPU (CUDA) would push to 30+ FPS for production
- No depth estimation
  - Single 2D camera can't determine real-world distance
- Largest bbox heuristic can fail
  - Closer bystander takes over when target walks away
- No re-identification (ReID)
  - Target leaving & returning gets assigned a new tracker ID
- No servo/motor integration
  - Direction messages are visual only; no physical PTZ control

Questions & Answers

Thank you for your attention!

Hye Ran Yoo 041145212 | Peng Wang 041107730

CST8508 – Machine Vision | April 2026